

Explore the network

Start here

WSL: start an exercise. Each command resets its scenario.

```
make networking      # follow a request
make operations      # find the fault
make automation      # repair through an API
make monitoring      # measure traffic and loss
```

Help: make task, make hint STEP=2, make solution STEP=2.

Lagoon probe (replace lagoontransit with pacifictransit for Pacific):

```
make enter NODE=probe01.bob1.lagoontransit.test
```

Inside a probe

```
ip -br addr          # my addresses
ip route get 198.51.100.10      # IPv4 gateway and source
ip -6 route get 2001:db8:100::10 # IPv6 gateway and source
dig @198.51.100.10 data.oceanresearch.test A +time=1 +tries=1
dig @2001:db8:100::10 data.oceanresearch.test AAAA +time=1 +tries=1
ping -4 -c 3 -W 1 198.51.100.10 # IPv4 reachability
ping -6 -c 3 -W 1 2001:db8:100::10 # IPv6 reachability
curl --fail --max-time 5 -4 http://data.oceanresearch.test/
curl --fail --max-time 5 -6 http://data.oceanresearch.test/
```

Look inside a router

In WSL: enter ReefNet Edge 01.

```
make enter NODE=edge01.bob1.reefnet.test
```

For Lagoon, use NODE=edge01.bob1.lagoontransit.test.

SR Linux CLI

Interfaces and routing sessions:

```
show interface brief
show network-instance default protocols bgp neighbor
show network-instance default protocols ospf neighbor
```

Customer prefix: BGP and active route table.

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
show network-instance default route-table ipv4-unicast prefix 198.51.100.0/24
```

On ReefNet Edge 01, what do we advertise to Lagoon? Which policies apply?

```
show network-instance default protocols bgp neighbor 192.0.2.2 advertised-routes ipv4
info from running network-instance default protocols bgp
info from running routing-policy
```

Inspect and change configuration

Change the Lagoon export policy from the CLI

After the investigation, inspect both the group override and BGP's inherited policy:

```
info from running network-instance default protocols bgp group lagoon-v4
info from running network-instance default protocols bgp
```

Stage the scoped change. It removes the group's override so it inherits EXPORT-BGP.

```
enter candidate
delete / network-instance default protocols bgp group lagoon-v4 export-policy
diff
```

Commit only if diff shows that single removal. Then verify the running config and test from the probe.

```
commit now
info from running network-instance default protocols bgp group lagoon-v4
```

Use `discard now` to cancel uncommitted edits. To restore the committed before-state, set the original attachment:

```
enter candidate
set / network-instance default protocols bgp group lagoon-v4 export-policy [ BLOCK-CUSTOMER-V4 ]
diff
commit now
```

Read through an API

In WSL: enter the operator workstation.

```
make enter NODE=operations01.bob1.reefnet.test
```

Run the commands below **inside operations01**.

Find a device in NetBox

```
TOKEN=$(cat /state/netbox-token)
NETBOX_API=http://netbox.bob1.reefnet.test:8080/api
curl -fsSG -H "Authorization: Bearer $TOKEN" \
  "$NETBOX_API/dcim/devices/" \
  --data-urlencode name=edge01.bob1.reefnet.test |
jq '{count, devices: [.results[] | {name, primary_ip4}]}'
```

Expect one device. Check its name and primary IP. Keep the token private.

Read the BGP configuration with gNMI

```
BGP_PATH='/network-instance[name=default]/protocols/bgp'
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSr11!' --skip-verify --encoding json_ietf \
  get --type config --path "$BGP_PATH"
```

Measure traffic

Change a BGP policy with gNMI

Still inside operations01: read the affected group and note its current policy.

```
BGP_PATH='/network-instance[name=default]/protocols/bgp'
GROUP="$BGP_PATH/group[group-name=lagoon-v4]"
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSr11!' --skip-verify --encoding json_ietf \
  get --type config --path "$GROUP"
```

Remove the group's policy override

After diagnosis: the group uses BLOCK-CUSTOMER-V4, causing the fault. Its parent uses EXPORT-BGP. Delete the override to inherit that policy.

```
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSr11!' --skip-verify --encoding json_ietf \
  set --delete "$GROUP/export-policy"
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSr11!' --skip-verify --encoding json_ietf \
  get --type config --path "$GROUP"
```

Verify routes and service from both probes. Use exit to return to WSL. Change and rollback: make solution SCENARIO=automation STEP=3.

Watch traffic and loss

In WSL: choose one load. Wait at least 30 seconds at each level.

```
make traffic-40mbit
make traffic-60mbit
make traffic-75mbit
make traffic-10mbit
make traffic-diagnostics # inspect counters and receiver loss
```

Grafana: http://localhost:3000, folder **ReefNet / BOB1**. Check the interface, direction and time window.

In WSL: make enter NODE=edge01.bob1.lagoontransit.test In its router CLI:

```
info from state interface ethernet-1/1 statistics
```

Compare out-discarded-packets at two times.

Leave with quit. In WSL: make enter NODE=service01.bob1.oceanresearch.test Inside the service node:

```
tail -n 15 /tmp/iperf-server.log # received Mbit/s and loss
```

Run experiments

Control the experiment

In **WSL**: run `make traffic-10mbit`, then `make test`. Start only when all four results pass. Try one fault at a time.

Experiment	Inject	Restore
Link failure	<code>make fault-link</code>	<code>make clear-link</code>
Routing failure	<code>make fault-routing</code>	<code>make clear-routing</code>

Observe, restore, then repeat `make test` after convergence. Investigation: `make hint SCENARIO=monitoring STEP=4` (link), `STEP=5` (routing).

Short burst

```
make traffic-stop          # stop continuous traffic first
```

For the rate-window comparison, wait five minutes. Then:

```
make traffic-burst-40mbit  # stops after five seconds
make traffic-status        # current load
```

Queries: `make hint SCENARIO=monitoring STEP=6`.

Stop and repeat

Keep notes on your findings. `make down` removes the lab and discards router changes.
Scenario commands and `make reset` recreate the initial state. `make clean` also deletes lab state and NetBox data.